

JWT Tokens: All user sessions were secured using signed JSON Web Tokens. Each token carries role-based access metadata and has a configurable expiration window.

Password handling: User passwords were hashed using BCrypt, providing protection against brute-force attacks and ensuring safe storage.

Docker hygiene: Each microservice included a `.dockerignore` file to exclude build artifacts, development dependencies, and sensitive files from the final image.

Kubernetes Secrets: Sensitive values such as database credentials, API tokens, and service-specific keys were stored and injected using native Kubernetes Secrets, ensuring they are not hardcoded or exposed in the version control system.

Observed metrics

As HealSphere scaled across microservices and environments, we tracked key metrics to validate performance and deployment efficiency.

- **Microservice startup time:** Most Spring Boot services initialise in 3–5 seconds, depending on database readiness and external config loading.
- **Jenkins CI build time:** Each microservice takes approximately 1 minute to build and containerise via Jenkins.
- **Deployment time:** End-to-end rollout using Ansible + kubectl takes around 30 seconds, including manifest application and pod spin-up.

CI/CD workflow: End-to-end automation

HealSphere's deployment pipeline is fully automated using GitHub, Jenkins, Docker, Ansible, and Kubernetes. Below is a step-by-step breakdown of how things move from code to production, based on the logic defined in the project's Jenkinsfile.

Code push to GitHub

- Developers modify the source code for individual microservices or the frontend.
- Once tested locally, changes are committed and pushed to the central GitHub repository.
- This Git operation serves as the primary trigger for the CI/CD pipeline.

GitHub webhook trigger

- GitHub is configured with a webhook that immediately notifies Jenkins of new commits or push events.
- This eliminates polling delays and ensures that builds are triggered as soon as code is pushed.

Jenkins Pipeline execution

- Upon receiving the webhook, Jenkins invokes the pipeline defined in the Jenkinsfile.
- Jenkins pulls the latest code and iterates over each

microservice listed in the `servicesToBuild` map.

- For each service, it:
 - Performs a Maven build (for Spring Boot services).
 - Executes Docker builds using multistage Dockerfiles.
 - Tags the resulting images (e.g., latest) and pushes them to Docker Hub.
- Jenkins also manages credentials (DockerHubCred), logs errors, and dispatches notifications depending on pipeline outcome.

Docker image packaging

- Each microservice is packaged into its own Docker image.
- Tagged images are uploaded to Docker Hub (kanang namespace), making them ready for deployment.
- This containerisation ensures consistent builds, isolated runtime environments, and faster deployments.

Ansible deployment to Kubernetes

- After images are published, Jenkins executes the `deploy-k8s.yml` Ansible playbook.
- Ansible uses the kubectl module to apply Kubernetes manifests stored in the `k8s/` directory.
- All services are deployed into a dedicated namespace (mindnotes) within the cluster.

Minikube as the staging environment

- A Minikube Kubernetes cluster is used for staging deployments, running either locally or on a controlled VM.
- Kubernetes handles pod scheduling, auto-restarts, scaling, service discovery, and internal load balancing.

Application availability

- Once deployment is complete, services are exposed via Kubernetes Ingress or NodePort (depending on environment setup).
- End users can access the application through a browser at the configured URL or IP, with traffic routed to the appropriate frontend and backend services.

Figure 2 shows the Jenkins Stage View for HealSphere's CI/CD pipeline. Each column represents a pipeline stage, including source checkout, microservice build, Docker image creation, Ansible-based Kubernetes deployment, and final cleanup. The green boxes indicate successful execution of each step, with precise timestamps and stage durations.

This visual view helps developers:

- Monitor pipeline health at a glance
- Identify stage-level bottlenecks (e.g., image push time)
- Confirm consistent deployment behaviour across builds

In the highlighted run (#10), the entire process from code checkout to Kubernetes deployment completes in just under 3 minutes, demonstrating a streamlined automation flow.